

imgsvg

Generated by Doxygen 1.16.1

Chapter 1

imgsvg — Raster Graphics Vectorizer

1.1 Introduction

A command-line utility designed to convert raster images into scalable and optimized vector graphics.

1.2 Processing Pipeline

The vectorization process consists of the following consecutive stages:

- **Resize:** Proportionally scales down images exceeding the maximum width.
- **KMeans:** Reduces image color depth through pixel clustering.
- **UnionFind:** Groups connected pixels of identical color and filters noise.
- **EdgeDetection:** Traces region boundaries to extract discrete polygons.
- **PathOptimization:** Reduces vertex count using the RDP algorithm.
- **BezierSmoothing:** Converts polygon edges into continuous Cubic Bezier curves.
- **SvgOutput:** Exports computed shapes into a formatted SVG file.

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Color	RGB color representation	??
Config	Global configuration settings	??
Image	Raw image data	??
ImageRegions	Pixels grouped into connected regions	??
PathNode	Single node in a vector path	??
Point	Integer coordinates of a point	??
Polygon	Closed path made of points and a color	??
QuantizedImage	Image after color quantization	??
Shape	A vector shape composed of path nodes	??
Vec2	2D vector with double precision	??

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

include/ bezier_smoothing.hpp	Converts polygon edges into smooth Bezier curves	??
include/ edge_detection.hpp	Extracts polygons from image regions	??
include/ kmeans.hpp	Reducer for image color depth using KMeans clustering	??
include/ mock.hpp	Mock functions for testing segmentation logic	??
include/ path_optimization.hpp	Simplifies polygons by removing redundant points	??
include/ resize.hpp	Scales down images exceeding the maximum width	??
include/ svg_output.hpp	Exports vector shapes to an SVG file	??
include/ types.hpp	Common data structures for image processing and vectorization	??
include/ union_find.hpp	Groups pixels into connected regions and filters noise	??
src/ main.cpp	Entry point for the CLI Raster Vectorizer tool	??

Chapter 4

Class Documentation

4.1 Color Struct Reference

RGB color representation.

```
#include <types.hpp>
```

Public Attributes

- `uint8_t r`
Red channel (0-255).
- `uint8_t g`
Green channel (0-255).
- `uint8_t b`
Blue channel (0-255).

4.1.1 Detailed Description

RGB color representation.

The documentation for this struct was generated from the following file:

- [include/types.hpp](#)

4.2 Config Struct Reference

Global configuration settings.

```
#include <types.hpp>
```

Public Attributes

- `std::string output_path`
Path where the output file will be saved.
- `int max_width`
Maximum allowed image width in pixels.
- `int colors`
Target number of colors for quantization.
- `int min_area`
Minimum pixel area for a region to be kept.
- `double tolerance`
Maximum distance for path simplification.
- `double corner_threshold`
Angle threshold for smoothing.
- `double smoothing`
Multiplier for curve smoothing intensity.
- `double stroke_width`
Width of the stroke in the output SVG.
- `bool verbose`
Enables detailed logging to stdout.

4.2.1 Detailed Description

Global configuration settings.

The documentation for this struct was generated from the following file:

- `include/types.hpp`

4.3 Image Struct Reference

Raw image data.

```
#include <types.hpp>
```

Public Attributes

- `int w`
Image width in pixels.
- `int h`
Image height in pixels.
- `uint8_t * data`
Pointer to interleaved RGB pixel data.

4.3.1 Detailed Description

Raw image data.

The documentation for this struct was generated from the following file:

- [include/types.hpp](#)

4.4 ImageRegions Struct Reference

Pixels grouped into connected regions.

```
#include <types.hpp>
```

Public Attributes

- `std::vector< std::vector< int > >` **pixel_regions**
2D grid of region IDs.
- `std::unordered_map< int, Color >` **region_colors**
Maps region IDs to colors.

4.4.1 Detailed Description

Pixels grouped into connected regions.

The documentation for this struct was generated from the following file:

- [include/types.hpp](#)

4.5 PathNode Struct Reference

Single node in a vector path.

```
#include <types.hpp>
```

Public Attributes

- [Vec2](#) **position**
Main point position.
- [Vec2](#) **control1** {}
First Bezier control point.
- [Vec2](#) **control2** {}
Second Bezier control point.
- bool [is_curve](#)
True if the segment leading to this node is a curve.

4.5.1 Detailed Description

Single node in a vector path.

4.5.2 Member Data Documentation

4.5.2.1 is_curve

```
bool PathNode::is_curve
```

Initial value:

```
{  
    false}
```

True if the segment leading to this node is a curve.

The documentation for this struct was generated from the following file:

- [include/types.hpp](#)

4.6 Point Struct Reference

Integer coordinates of a point.

```
#include <types.hpp>
```

Public Attributes

- **int x**
Horizontal coordinate.
- **int y**
Vertical coordinate.

4.6.1 Detailed Description

Integer coordinates of a point.

The documentation for this struct was generated from the following file:

- [include/types.hpp](#)

4.7 Polygon Struct Reference

Closed path made of points and a color.

```
#include <types.hpp>
```


Public Attributes

- `std::vector< Point > points`
Ordered list of vertices.
- `Color color`
Fill color of the polygon.

4.7.1 Detailed Description

Closed path made of points and a color.

The documentation for this struct was generated from the following file:

- `include/types.hpp`

4.8 QuantizedImage Struct Reference

[Image](#) after color quantization.

```
#include <types.hpp>
```

Public Attributes

- `int w`
[Image](#) width in pixels.
- `int h`
[Image](#) height in pixels.
- `std::vector< int > pixel_labels`
Cluster index for each pixel.
- `std::vector< Color > palette`
List of colors used in the image.

4.8.1 Detailed Description

[Image](#) after color quantization.

The documentation for this struct was generated from the following file:

- `include/types.hpp`

4.9 Shape Struct Reference

A vector shape composed of path nodes.

```
#include <types.hpp>
```

Public Attributes

- `std::vector< PathNode > nodes`
Sequence of points and curves defining the outline.
- `Color color`
Fill color of the shape.

4.9.1 Detailed Description

A vector shape composed of path nodes.

The documentation for this struct was generated from the following file:

- `include/types.hpp`

4.10 Vec2 Struct Reference

2D vector with double precision.

```
#include <types.hpp>
```

Public Attributes

- `double x`
Horizontal component.
- `double y`
Vertical component.

4.10.1 Detailed Description

2D vector with double precision.

The documentation for this struct was generated from the following file:

- `include/types.hpp`

Chapter 5

File Documentation

5.1 include/bezier_smoothing.hpp File Reference

Converts polygon edges into smooth Bezier curves.

```
#include <vector>
#include "types.hpp"
```

Functions

- `std::vector< Shape > Stage::BezierSmoothing::process (const std::vector< Polygon > &polygons, const Config &config)`

Converts polygons into smooth vector shapes.

5.1.1 Detailed Description

Converts polygon edges into smooth Bezier curves.

5.1.2 Function Documentation

5.1.2.1 process()

```
std::vector< Shape > Stage::BezierSmoothing::process (
    const std::vector< Polygon > & polygons,
    const Config & config)
```

Converts polygons into smooth vector shapes.

Calculates the angle at each polygon point. If the angle is larger than the corner threshold, it creates a smooth curve. If the angle is smaller, it keeps a sharp corner.

Parameters

<i>polygons</i>	A list of input polygons.
<i>config</i>	Settings for smoothing, like corner threshold and optimize factor.

Returns

std::vector<Shape> List of shapes made of Bezier curves and straight lines.

5.2 bezier_smoothing.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <vector>
00009 #include "types.hpp"
00010
00011 namespace Stage::BezierSmoothing {
00012
00024 std::vector<Shape> process(const std::vector<Polygon>& polygons,
00025                           const Config& config);
00026
00027 } // namespace Stage::BezierSmoothing

```

5.3 include/edge_detection.hpp File Reference

Extracts polygons from image regions.

```
#include "types.hpp"
```

Functions

- std::vector< [Polygon](#) > [Stage::EdgeDetection::process](#) (const [ImageRegions](#) ®ions, const [Config](#) &config)

Traces region boundaries to create polygons.

5.3.1 Detailed Description

Extracts polygons from image regions.

5.3.2 Function Documentation

5.3.2.1 process()

```
std::vector< Polygon > Stage::EdgeDetection::process (
    const ImageRegions & regions,
    const Config & config)
```

Traces region boundaries to create polygons.

Iterates through pixels to find edges between different regions. Follows the perimeter of each region and stores it as a list of points.

Parameters

<i>regions</i>	Input grid of region IDs and their colors.
<i>config</i>	Settings containing minimum area filter.

Returns

std::vector<Polygon> List of polygons representing region outlines.

5.4 edge_detection.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "types.hpp"
00009
00010 namespace Stage::EdgeDetection {
00011
00022 std::vector<Polygon> process(const ImageRegions& regions, const Config& config);
00023
00024 } // namespace Stage::EdgeDetection
```

5.5 include/kmeans.hpp File Reference

Reducer for image color depth using KMeans clustering.

```
#include "types.hpp"
```

Functions

- [QuantizedImage Stage::KMeans::process](#) (const [Image](#) &image, const [Config](#) &config)
Groups image pixels into a fixed number of color clusters.

5.5.1 Detailed Description

Reducer for image color depth using KMeans clustering.

5.5.2 Function Documentation

5.5.2.1 process()

```
QuantizedImage Stage::KMeans::process (
    const Image & image,
    const Config & config)
```

Groups image pixels into a fixed number of color clusters.

Randomly initializes cluster centers and iteratively updates them by assigning pixels to the nearest color and recalculating the mean color.

Parameters

<i>image</i>	Input raw image data.
<i>config</i>	Settings containing the target number of colors.

Returns

[QuantizedImage Image](#) data with pixels mapped to a color palette.

5.6 kmeans.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "types.hpp"
00009
00010 namespace Stage::KMeans {
00011
00022 QuantizedImage process(const Image& image, const Config& config);
00023
00024 } // namespace Stage::KMeans
```

5.7 include/mock.hpp File Reference

Mock functions for testing segmentation logic.

```
#include "types.hpp"
```

Functions

- [ImageRegions mock_segmentation](#) (const [Image](#) &img)
Splits an image into two horizontal halves.
- [ImageRegions mock_segmentation_geom](#) (const [Image](#) &img)
Creates a circular and a rectangular region in the image.

5.7.1 Detailed Description

Mock functions for testing segmentation logic.

5.7.2 Function Documentation

5.7.2.1 mock_segmentation()

```
ImageRegions mock_segmentation (
    const Image & img)
```

Splits an image into two horizontal halves.

Parameters

<i>img</i>	Input image to split.
------------	-----------------------

Returns

[ImageRegions](#) Two regions (top and bottom) with red and blue colors.

5.7.2.2 mock_segmentation_geom()

```
ImageRegions mock_segmentation_geom (
    const Image & img)
```

Creates a circular and a rectangular region in the image.

Parameters

<i>img</i>	Input image.
------------	--------------

Returns

[ImageRegions](#) Regions representing a background, a circle and a box.

5.8 mock.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #include "types.hpp"
00007
00014 ImageRegions mock_segmentation(const Image& img);
00015
00022 ImageRegions mock_segmentation_geom(const Image& img);
```

5.9 include/path_optimization.hpp File Reference

Simplifies polygons by removing redundant points.

```
#include <vector>
#include "types.hpp"
```

Functions

- `std::vector< Polygon > Stage::PathOptimization::process` (const std::vector< Polygon > &polygons, const Config &config)
Reduces the number of points in polygons.

5.9.1 Detailed Description

Simplifies polygons by removing redundant points.

5.9.2 Function Documentation

5.9.2.1 process()

```
std::vector< Polygon > Stage::PathOptimization::process (
    const std::vector< Polygon > & polygons,
    const Config & config)
```

Reduces the number of points in polygons.

Uses the RDP algorithm to remove points that deviate less than the specified tolerance from a straight line.

Parameters

<i>polygons</i>	List of input polygons.
<i>config</i>	Settings containing the distance tolerance.

Returns

std::vector<Polygon> Simplified polygons with fewer vertices.

5.10 path_optimization.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <vector>
00009 #include "types.hpp"
00010
00011 namespace Stage::PathOptimization {
00012
00023 std::vector<Polygon> process(const std::vector<Polygon>& polygons,
00024                             const Config& config);
00025
00026 } // namespace Stage::PathOptimization
```


5.11 include/resize.hpp File Reference

Scales down images exceeding the maximum width.

```
#include "types.hpp"
```

Functions

- void [Stage::Resize::process](#) ([Image](#) &image, const [Config](#) &config)
Proportionally resizes an image if it exceeds the maximum width.

5.11.1 Detailed Description

Scales down images exceeding the maximum width.

5.11.2 Function Documentation

5.11.2.1 process()

```
void Stage::Resize::process (
    Image & image,
    const Config & config)
```

Proportionally resizes an image if it exceeds the maximum width.

Modifies the image dimensions and pixel data in place. If the image width is smaller than or equal to the maximum width, no changes are made.

Parameters

<i>image</i>	The image data to be resized.
<i>config</i>	Settings containing the maximum allowed width.

5.12 resize.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "types.hpp"
00009
00010 namespace Stage::Resize {
00011
00021 void process(Image& image, const Config& config);
00022
00023 } // namespace Stage::Resize
```

5.13 include/svg_output.hpp File Reference

Exports vector shapes to an SVG file.

```
#include <vector>
#include "types.hpp"
```

Functions

- void [Stage::SvgOutput::process](#) (const std::vector< [Shape](#) > &shapes, int width, int height, const [Config](#) &config)

Writes shapes into a formatted SVG file.

5.13.1 Detailed Description

Exports vector shapes to an SVG file.

5.13.2 Function Documentation

5.13.2.1 process()

```
void Stage::SvgOutput::process (
    const std::vector< Shape > & shapes,
    int width,
    int height,
    const Config & config)
```

Writes shapes into a formatted SVG file.

Converts internal shape representations into SVG path elements.

Parameters

<i>shapes</i>	List of Bezier curves and lines to export.
<i>width</i>	Canvas width.
<i>height</i>	Canvas height.
<i>config</i>	Settings containing the output file path.

5.14 svg_output.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <vector>
00009 #include "types.hpp"
00010
00011 namespace Stage::SvgOutput {
00012
00023 void process(const std::vector<Shape>& shapes, int width, int height,
00024             const Config& config);
00025
00026 } // namespace Stage::SvgOutput
```

5.15 include/types.hpp File Reference

Common data structures for image processing and vectorization.

```
#include <cstdint>
#include <string>
#include <unordered_map>
#include <vector>
```

Classes

- struct [Config](#)
Global configuration settings.
- struct [Color](#)
RGB color representation.
- struct [Image](#)
Raw image data.
- struct [QuantizedImage](#)
[Image](#) after color quantization.
- struct [ImageRegions](#)
Pixels grouped into connected regions.
- struct [Point](#)
Integer coordinates of a point.
- struct [Polygon](#)
Closed path made of points and a color.
- struct [Vec2](#)
2D vector with double precision.
- struct [PathNode](#)
Single node in a vector path.
- struct [Shape](#)
A vector shape composed of path nodes.

5.15.1 Detailed Description

Common data structures for image processing and vectorization.

5.16 types.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <cstdint>
00009 #include <string>
00010 #include <unordered_map>
00011 #include <vector>
00012
00016 struct Config {
00017     std::string output_path;
00018     int max_width;
00019     int colors;
```

```

00020     int min_area;
00021     double tolerance;
00022     double corner_threshold;
00023     double smoothing;
00024     double stroke_width;
00025     bool verbose;
00026 };
00027
00031 struct Color {
00032     uint8_t r;
00033     uint8_t g;
00034     uint8_t b;
00035 };
00036
00040 struct Image {
00041     int w;
00042     int h;
00043     uint8_t* data;
00044 };
00045
00049 struct QuantizedImage {
00050     int w;
00051     int h;
00052     std::vector<int> pixel_labels;
00053     std::vector<Color> palette;
00054 };
00055
00059 struct ImageRegions {
00060     std::vector<std::vector<int>> pixel_regions;
00061     std::unordered_map<int, Color> region_colors;
00062 };
00063
00067 struct Point {
00068     int x;
00069     int y;
00070 };
00071
00075 struct Polygon {
00076     std::vector<Point> points;
00077     Color color;
00078 };
00079
00083 struct Vec2 {
00084     double x;
00085     double y;
00086 };
00087
00091 struct PathNode {
00092     Vec2 position;
00093     Vec2 control1{};
00094     Vec2 control2{};
00095     bool is_curve{
00096         false};
00097 };
00098
00102 struct Shape {
00103     std::vector<PathNode>
00104         nodes;
00105     Color color;
00106 };

```

5.17 include/union_find.hpp File Reference

Groups pixels into connected regions and filters noise.

```
#include "types.hpp"
```

Functions

- [ImageRegions Stage::UnionFind::process](#) (const [QuantizedImage](#) &image, const [Config](#) &config)
Identifies connected components of the same color.

5.17.1 Detailed Description

Groups pixels into connected regions and filters noise.

5.17.2 Function Documentation

5.17.2.1 process()

```
ImageRegions Stage::UnionFind::process (
    const QuantizedImage & image,
    const Config & config)
```

Identifies connected components of the same color.

Groups adjacent pixels with the same label into regions. Small regions are merged with their largest neighbors to reduce noise.

Parameters

<i>image</i>	Quantized image with pixel labels.
<i>config</i>	Settings containing the minimum area threshold.

Returns

[ImageRegions](#) Data structure containing the identified regions and colors.

5.18 union_find.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "types.hpp"
00009
00010 namespace Stage::UnionFind {
00011
00022 ImageRegions process(const QuantizedImage& image, const Config& config);
00023
00024 } // namespace Stage::UnionFind
```

5.19 src/main.cpp File Reference

Entry point for the CLI Raster Vectorizer tool.

```
#include <boost/filesystem.hpp>
#include <boost/program_options.hpp>
#include <iostream>
#include <print>
#include "stb_image.h"
#include "bezier_smoothing.hpp"
#include "edge_detection.hpp"
```

```
#include "kmeans.hpp"
#include "mock.hpp"
#include "path_optimization.hpp"
#include "resize.hpp"
#include "svg_output.hpp"
#include "types.hpp"
#include "union_find.hpp"
```

Functions

- `int main (int argc, char *argv[])`
Main function handling CLI arguments and program flow.

5.19.1 Detailed Description

Entry point for the CLI Raster Vectorizer tool.